

Reducing Token, Latency, and Compute Costs of a Recurring LLM Market-Analysis Task by Moving Deterministic Work into Code

Aoba Three (research@aobathree.com)

Prepared with the assistance of ChatGPT, Claude, and Cursor · August 2, 2026

Artifacts: <https://github.com/aobathree/llm-interpreter-compiler-modes>

Abstract—A recurring market-analysis task run through an LLM agent repeats work that never changes: each run re-fetched raw price data and re-derived technical indicators inside the model's context. We restructured the task so that all deterministic work—data retrieval, indicator computation, formatting—is performed by conventional code emitting a compact digest, and the LLM reads only that digest. This paper reports the design considerations, three successive implementations, and their measurements, which show substantial reductions in context consumption, latency, and CPU usage, and which surfaced operational constraints—server-side rate limiting, resource costs hidden behind unchanged wall-clock time—that only measurement could reveal. The resulting pipeline runs unattended, hourly, on serverless infrastructure at effectively zero cost.

1 Problem

The motivating task is a recurring trading-condition digest—price, RSI, MACD sign, position relative to moving averages, volume pattern, and ATR—for currency pairs listed on the bitbank cryptocurrency exchange. Performed by an LLM agent directly, each run fetched several hundred raw OHLCV candles per pair into the model's context and computed indicators in conversation. This had four recurring costs: (i) context consumption exceeding 10^4 tokens per run, growing linearly with the amount of history fetched; (ii) minute-scale latency dominated by model inference; (iii) arithmetic-error risk in manually derived indicators, adding verification overhead; and (iv) non-reproducibility, since model output varies across runs. None of these costs purchases anything after the first run: the procedure itself is stable from day to day.

2 Design Considerations

The workload divides into three parts with different requirements. Deterministic operations (retrieval, aggregation, indicator arithmetic, formatting) can be fixed as ordinary code. Run-specific values (the target instruments) must remain parameters rather than being hard-coded from one observed run. Genuinely semantic work (a one-line commentary on the digest) remains an LLM responsibility, but operates on the digest text alone.

Three further requirements shaped the design. First, an output contract: a fixed three-line digest per instrument, readable by both humans and LLMs, stable enough to compare byte-for-byte across implementations (golden testing). Second, guards: indicators are computed from confirmed candles only, per-instrument failures are contained as single error lines rather than aborting the run, and external-API concurrency is bounded. Third, an escape hatch: when assumptions break—API changes, unexpected data shapes—the task falls back to interactive analysis, and the code is revised.

3 Implementation

Three implementations were built, each preserving the output contract. (1) A Python script (standard library only) fetching daily and hourly candles through the exchange's CLI and computing all indicators. (2) A native Rust binary that issues all fetches in parallel under a semaphore capping in-flight requests at 16. (3) A Rust variant that calls the exchange REST API directly with pooled TLS connections—four HTTP requests per instrument—instead of spawning CLI subprocesses. Output equivalence across (1)–(3) was verified byte-for-byte.

For production, the digest core was extracted into a library and deployed as an hourly serverless job: a scheduler invokes a 256 MB arm64 Rust Lambda function, which reads its instrument configuration from a single DynamoDB record, generates the digest, and posts it to a Discord webhook, chunked to the platform's 2,000-character message limit. A token-authenticated configuration API and a static single-page GUI allow the instrument list to be changed at runtime; changes take effect at the next scheduled run without redeployment. No always-on infrastructure is used.

4 Results

Stage	Implementation	LLM reads	Latency
0	LLM in-context computation	$>10^4$ tokens	minutes
1	Python script (sequential)	~ 10 lines	0.6 s
2	Rust, parallel fetch	~ 10 lines	0.11 s
3	Rust, direct REST API	~ 10 lines	0.11 s

Table 1. Per-run cost by stage (3 instruments; medians). Stages 1–3 produce byte-identical digests.

Latency. Table 1 shows the ladder. The stage-1→2 speedup ($5.5\times$) comes almost entirely from eliminating interpreter startup and overlapping six network fetches; indicator arithmetic is sub-millisecond in every implementation, and the workload is network-bound.

Scale. Extending from 3 to all 62 listed pairs (124 parallel fetches, >300 HTTP requests) surfaced a constraint invisible at small scale: with unbounded parallelism, runs above ~ 30 pairs stalled intermittently for 15–26 s with no error returned, consistent with token-bucket rate limiting. With concurrency capped at 16, all 62 pairs complete in ~ 1.2 s, matching the estimate $124 \text{ fetches} \div 16 \approx 8 \text{ waves} \times 0.15 \text{ s}$.

Resources. Stages 2 and 3 are nearly indistinguishable by wall-clock time, which suggested that bypassing the CLI layer was not worthwhile. Process-tree accounting (Windows Job Objects, which capture all short-lived child processes) showed otherwise (Table 2): the subprocess design spends $42\times$ more CPU and two orders of magnitude more processes and page faults, hidden behind network waiting on a multi-core host. On CPU-constrained targets—small containers, low-end VPS instances—this difference re-emerges as wall-clock time.

Metric (62 pairs)	CLI subproc.	Direct API	Ratio
Wall-clock time	1.1–1.7 s	1.1 s	$\sim 1.2\times$
CPU time (tree)	1.97 s	0.05 s	$42\times$
Processes	125	1	$125\times$
Page faults	3.2×10^5	6.5×10^3	$50\times$
Peak memory (job)	30 MB	20 MB	$1.5\times$

Table 2. Resource consumption via Job Object accounting (medians of 3 runs).

Operation. The deployed function generates and delivers a digest in ~ 0.2 s per hourly invocation within the 256 MB memory class. Monthly operating cost is effectively zero (720 invocations, single-record database reads, no always-on resources). A runtime configuration change made through the GUI was observed to take effect at the immediately following run.

5 Lessons

Two findings contradicted our working hypotheses and were only established by measurement. First, subprocess overhead was assumed to dominate stage 2; in fact the vendor CLI is a thin native binary and its cost is invisible at wall-clock granularity—while an initial direct-API build was *slower* than the CLI design until an HTTP connection-pool default (one idle connection per host, forcing TLS re-handshakes on every wave) was corrected. Second, equal wall-clock time concealed the $42\times$ CPU difference of Table 2: "no faster" and "no lighter" are independent claims, and the dimension that matters depends on the deployment target. More generally, scale changed the problem qualitatively—rate limiting appears only beyond ~ 30 instruments—so guards discovered by measurement, not the initial design, made the pipeline production-ready.

6 Conclusion

Moving the deterministic majority of a recurring LLM-assisted analysis into conventional code reduced per-run context consumption by three orders of magnitude, latency from minutes to ~ 0.1 s, and CPU consumption by $42\times$ relative to a subprocess-based variant, while making outputs reproducible and testable. The result runs unattended, hourly, at effectively zero cost, with the LLM retained only where it adds value: reading a ten-line digest and commenting on it. On the strength of these measurements, the digest command has been proposed for inclusion in the exchange's official CLI (bitbankinc/bitbank-lab-cli, Issue #21), where its deterministic computation naturally belongs. All code, benchmark procedures, and raw measurements are available in the artifacts repository.